

Migração Fase 1: Supabase → API AdonisJS própria

Kover Manutenção — plano de implementação

Contexto

O sistema "Kover Manutenção" hoje vive em dois projetos desconectados:

- **Koverman-revisao1** (backend AdonisJS): API AdonisJS 5 (JS puro) com SQLite, contendo apenas 2 tabelas de exemplo/placeholder (`tecnicos` , `ordens_servicos`) criadas como demonstração de padrão, sem dados reais.
- **kovermanutencao-main**: o app React/TanStack Start real, completo, hoje 100% dependente do Supabase (Postgres + Auth) e, para push, do Firebase.

O usuário pediu para migrar o frontend do Supabase para este backend local. O escopo real é grande: 32 tabelas, ~38 arquivos de lógica de servidor, 4 fluxos de autenticação distintos (admin/coordenador/produção via Supabase Auth + técnico via sessão custom por cookie). Após alinhamento, ficou definido:

- **Fase 1 agora**: módulos essenciais — autenticação (os 4 perfis), técnicos, setores, categorias, máquinas, materiais/estoque, ordens de serviço externas e internas, log de auditoria.
- **Fases seguintes** (fora deste plano): indicadores/dashboards, preventivas, compras, notificações, push Firebase.
- **Banco**: manter SQLite.
- **Auth**: replicar os 4 fluxos fielmente (não simplificar).
- **Frontend**: religar já nesta fase para os módulos da Fase 1 (os módulos fora do escopo continuam no Supabase por enquanto — app fica num estado híbrido temporário).

As tabelas/regras abaixo foram levantadas lendo o schema real do Supabase (`supabase/migrations/*.sql` + `types.ts`) e a lógica de negócio real (`src/lib/*.functions.ts`) do projeto `kovermanutencao-main` — não são um redesenho, são a réplica fiel do sistema existente.

Decisão importante: as tabelas placeholder atuais serão substituídas

`tecnicos` e `ordens_servicos` (schema atual do Adonis) não têm relação nenhuma com as tabelas reais `technicians` / `work_orders` do Supabase — foram só um exemplo de padrão, conforme o próprio README explica. Como não há dado real nelas (confirmado, tabela vazia), a Fase 1 vai:

1. `node ace migration:rollback` das 2 migrations placeholder (remove `tecnicos` / `ordens_servicos`).
2. Apagar `app/Models/Tecnico.js` , `app/Models/OrdemServico.js` , `app/Controllers/Http/TecnicosController.js` , `app/Controllers/Http/OrdensServicoController.js` , `app/Validators/CreateTecnico.js` , `app/Validators/CreateOrdemServico.js` , e as rotas correspondentes em `start/routes.js` .
3. Recriar os módulos reais (`technicians` , `work_orders` , etc.) do zero, seguindo os MESMOS padrões de código já estabelecidos (`BaseModel` com `$addColumn` / `$addRelation` manual, `dateTimeColumn.js` , `Validators` com `schema` / `messages` , paginação via `.paginate()` , `.preload()` para relações).

Decisões de nomenclatura e tipos (para reduzir risco de tradução)

- **Nomes de tabela/coluna em inglês**, iguais ao Supabase (`technicians` , `work_orders` , `technician_id` , etc.) — não traduzir para português. Elimina uma camada inteira de risco de erro de tradução contra a extensa pesquisa de schema/regras já feita, e minimiza o quanto o frontend precisa mudar.
- **IDs como inteiro autoincrement** (`table.increments('id')`), não UUID — segue o padrão já usado em `Tecnico` / `OrdemServico` deste projeto.

- **Enums via** `table.enu(...)` (Knex/SQLite emula com `CHECK`), exatamente como já é feito em `170000000002_ordens_servicos_schema.js`.
- **Colunas decimais** via `table.decimal(col, 18, 4)` uniformemente — simplificação segura, sem mudança de comportamento visível.
- Tabelas de fases futuras (`suppliers`, `stock_receipts`, `maintenance_plans`) — colunas que apontariam para elas ficam de fora por ora (YAGNI).
- **Correção deliberada:** operações de estoque hoje no Supabase usam concorrência otimista sem transação real. Na reconstrução, usar `Database.transaction()` do Lucid — mais robusto, mesmo comportamento observável.

Arquitetura de autenticação

Nenhum pacote de auth está de fato registrado hoje (`@adonisjs/hash` existe em disco mas não está no `.adonisrc.json`; não há `@adonisjs/auth`, `@adonisjs/session`, `bcrypt` nem `jsonwebtoken` instalados).

Novas dependências: `bcrypt` (hash de senha admin/coordenador/produção via `@adonisjs/hash`), `jsonwebtoken` (JWT desses 3 perfis). Hash do código numérico do técnico (PBKDF2) e token de sessão (SHA-256) usam o módulo `crypto` nativo do Node — sem pacote novo.

Dois mecanismos de auth em paralelo (fiel ao sistema real):

1. **JWT para admin/coordenador/produção** — `app/Services/Jwt.js` (assina/verifica via `jsonwebtoken`), `app/Middleware/Auth.js` (named `auth`: lê `Authorization: Bearer <token>`, carrega `ctx.user` + roles de `user_roles`, 401 se inválido/banido). O frontend já tem exatamente esse padrão implementado em `src/integrations/supabase/auth-attacher.ts` — trocar a origem do token é a única mudança estrutural necessária ali.
2. **Sessão por cookie para técnico** — `app/Middleware/TechAuth.js` (named `techAuth`: lê cookie `tech_sid`, hash SHA-256, busca em `technician_sessions`, aplica TTL absoluto de 8h + timeout de inatividade de 2h com sliding window, valida `status='ativo'`, popula `ctx.technician`) — replica exatamente `requireTechnicianFromCookie()` do frontend atual.

`app/Services/Authorization.js` centraliza checagem de papel (`assertRole(ctx, ['admin'])`) — hoje isso é reimplementado ad-hoc em cada arquivo no frontend; centralizar aqui é uma limpeza de baixo risco.

`app/Services/AuditLog.js` — helper único `log(ctx, {action, entityType, entityId, details})` usado por todos os controllers que hoje geram `audit_logs` no Supabase.

Tabelas da Fase 1 (ordem de criação = ordem de FK)

Cada uma vira uma migration + Model seguindo o padrão de `Tecnico.js` / `OrdemServico.js`:

1. `sectors` — name, active
2. `categories` — name, active, created_by/updated_by
3. `users` — substitui `auth.users` + `profiles` do Supabase num só lugar: email, username (nullable, único, para produção), `password_hash`, `full_name`, cargo, status(ativo/inativo), `matricula`, `sector_id` FK, turno(manha/tarde/noite), `email_confirmed`, `banned`
4. `user_roles` — `user_id` FK, role enum(admin/coordenador/producao), único(`user_id,role`)
5. `technicians` — name, `code_hash`, status(ativo/inativo), created_by/updated_by FK→users
6. `technician_sessions` — `technician_id` FK, token_hash único, expires_at, last_activity_at, ip, user_agent, revoked_at
7. `machines` — name, sector(texto livre), status, code(único opcional), fabricante, modelo, serial_number, acquired_at
8. `materials` — code único, name, category_id FK, unit enum(20 valores — UN/PC/CJ/PAR/MM/CM/M/M2/M3/POL/KG/G/L/ML/CX/RL/SC/BD/LT/PCT), quantity, unit_value, min_quantity, notes,

status(ativo/inativo), created_by/updated_by FK→users

9. `stock_movements` — material_id FK, type enum(retirada/ajuste_entrada/ajuste_saida/inventario_inicial/recebimento), quantity, unit_value_snapshot, total_value, technician_id FK nullable, performed_by_user FK→users nullable, reason, notes, nf_number, oc_number, justification, internal_wo_id FK nullable, work_order_id FK nullable
10. `work_orders` — number(único, gerado OS-000123 após insert), requester_user_id FK→users, requester_name/sector_id/sector_name/turno, machine_id FK, machine_code/name/sector, priority enum(baixa/media/alta/emergencial), description, status enum(aguardando_atendimento/aceita/em_atendimento/pausada/aguardando_avaliacao/encerrada/reaberta/cancelada/concluido), opened_at, technician_id FK nullable, accepted_at/started_at/finished_at, service_type enum nullable, technician_comment, final_comment, evaluated_at, evaluation_result, evaluation_comment, reopen_reason/reopened_at/reopen_count, occurrence_type enum nullable, machine_stopped, paused_at, total_paused_seconds, maintenance_type enum(corretiva/preventiva/preditiva/inspecao) default corretiva, due_at, estimated_minutes, validated_at/validated_by/validation_comment
11. `work_order_materials` — work_order_id FK, material_id FK, quantity, unit_value_snapshot, total_value, technician_id FK nullable
12. `work_order_comments` — work_order_id FK, technician_id FK nullable, technician_name, comment
13. `work_order_events` — work_order_id FK, technician_id FK nullable, technician_name, event_type, message, details(json)
14. `work_order_participants` — work_order_id FK, technician_id FK, technician_name, first_joined_at, last_active_at, único(work_order_id,technician_id)
15. `work_order_pauses` — work_order_id FK, technician_id FK nullable, reason, started_at, ended_at
16. `internal_work_orders` — code(único, gerado OSI-000123), origin default 'manutencao', machine_id FK, sector, technician_id FK, opened_at, service_type enum(mecanico/eletrico/pneumatico/hidraulico/electronico/instrumentacao/lubrificacao/soldagem/outro), description, technician_comment, started_at/finished_at nullable, status enum(aberta/concluida/aceita/em_atendimento/pausada) default concluida, accepted_at, paused_at, total_paused_seconds
17. `internal_wo_materials` — internal_wo_id FK, material_id FK, quantity, unit_value_snapshot, total_value
18. `internal_wo_pauses` — internal_wo_id FK, technician_id FK nullable, reason, started_at, ended_at
19. `audit_logs` — action, actor_user_id FK→users nullable, actor_technician_id FK→technicians nullable, actor_role, entity_type, entity_id, ip, user_agent, details(json)

Controllers e regras de negócio a portar (fielmente)

- **AuthController:** login (email/username+senha→JWT), `claimFirstAdmin` (bootstrap: primeiro usuário vira admin se `user_roles` estiver vazio), `me`.
- **UsersController** (admin-only, espelha `auth/pcm.functions.ts`): listar, criar (admin/coordenador exigem email+senha; produção exige username+setor+turno, email sintético `username@producao.local`), atualizar, `setRole` (bloqueia auto-revogar admin e remover o último admin).
- **TechAuthController:** `listActiveTechnicians` (público), `login` (verifica code vs code_hash via PBKDF2, cria sessão, seta cookie tech_sid httpOnly), `logout` (revoga sessão), `me`.
- **TechniciansController** (admin-only CRUD): criar (hash do código), status, atualizar (novo código opcional).
- **MachinesController:** listar (qualquer autenticado), criar/atualizar (admin-only), código único case-insensitive.
- **MaterialsController:** listagem dupla (admin vê unit_value, técnico não), criação sempre com quantity=0, atualização valida compatibilidade decimal ao trocar unidade, bulkImport com validação linha-a-linha + dry-run, delete admin-only com cascata em stock_movements.
- **StockController:** adjust (ajuste_entrada exige nf/oc/justificativa ≥ 10 chars; ajuste_saida não pode negativar; tudo em transação).

- **WithdrawalsController** (técnico-only): criar retirada (decrementa estoque + audit, em transação), histórico próprio.
- **WorkOrdersController**: criar (bloqueia nova OS se o solicitante já tem uma em aguardando_avaliacao; valida máquina pertence ao setor informado; prioridade escala pra crítica se emergencial/machine_stopped), listar, evaluate (aprova→encerrada; rejeita→reaberta e reseta técnico/datas).
- **WorkOrdersTechController** (técnico-only, sem "dono" único): accept/start, takeOver, comment, updateExecution, addMaterial (transação), finish (bloqueia se preventiva com checklist obrigatório pendente), pause/resume (acumula total_paused_seconds), updateChecklistItem. Toda ação grava em work_order_events + audit_logs + work_order_participants (upsert).
- **InternalWorkOrdersController** (técnico-only): criar já concluída com materiais consumidos atomicamente (transação substitui o rollback manual do código original), listar/ver só as próprias.

Rotas (/api/v1)

```

POST    /auth/login                               AuthController.login
POST    /auth/claim-first-admin                 AuthController.claimFirstAdmin
GET     /auth/me                               [auth] AuthController.me
Route.resource('users', ...) [auth] UsersController (admin dentro do controller)
PATCH  /users/:id/role                         [auth] UsersController.setRole

GET     /tech/technicians                      TechAuthController.listActive
POST    /tech/login                          TechAuthController.login
POST    /tech/logout                        [techAuth] TechAuthController.logout
GET     /tech/me                            [techAuth] TechAuthController.me

Route.resource('technicians', ...) [auth] TechniciansController
Route.resource('machines', ...) [auth] MachinesController
Route.resource('materials', ...) [auth] MaterialsController
POST    /materials/bulk-import                [auth] MaterialsController.bulkImport
POST    /stock/adjust                        [auth] StockController.adjust
GET     /stock/movements                     [auth] StockController.index

POST    /withdrawals                         [techAuth] WithdrawalsController.store
GET     /withdrawals/mine                    [techAuth] WithdrawalsController.mine

Route.resource('work-orders', ...) [auth] WorkOrdersController
PATCH  /work-orders/:id/evaluate             [auth] WorkOrdersController.evaluate
PATCH  /work-orders/:id/accept               [techAuth] WorkOrdersTechController.accept
PATCH  /work-orders/:id/take-over            [techAuth] WorkOrdersTechController.takeOver
POST    /work-orders/:id/comments             [techAuth] WorkOrdersTechController.addComment
PATCH  /work-orders/:id/execution            [techAuth] WorkOrdersTechController.updateExecution
POST    /work-orders/:id/materials            [techAuth] WorkOrdersTechController.addMaterial
PATCH  /work-orders/:id/finish               [techAuth] WorkOrdersTechController.finish
PATCH  /work-orders/:id/pause                [techAuth] WorkOrdersTechController.pause
PATCH  /work-orders/:id/resume               [techAuth] WorkOrdersTechController.resume
PATCH  /work-orders/:id/checklist/:itemId    [techAuth] WorkOrdersTechController.updateChecklistItem

Route.resource('internal-work-orders', ...) [techAuth] InternalWorkOrdersController

```

Religação do frontend (kovermanutencao-main)

- Novo `src/lib/api-client.ts`: wrapper de fetch contra `VITE_API_URL` (default `http://localhost:3333/api/v1`), injeta `Authorization: Bearer` (localStorage) e usa `credentials: 'include'` para o cookie `tech_sid` (CORS já está `credentials: true` em `config/cors.js`).

- `auth-attacher.ts` / `auth-middleware.ts` : trocar a extração de sessão Supabase pelo token JWT próprio guardado no client.
- Cada arquivo em `src/lib/*.functions.ts` dos módulos da Fase 1 (technicians, team, auth/tech, auth/pcm, machines, materials, stock, withdrawals, work-orders, wo-tech, wo-producao, internal-wo) — trocar as chamadas `supabase.from(...)` pelas chamadas equivalentes ao api-client, mantendo os mesmos nomes de função exportados.
- Módulos fora da Fase 1 (indicadores, preventiva, compras, notificações, push) permanecem no Supabase — app fica em estado híbrido até as próximas fases.

Verificação

Depois de cada módulo implementado, `npm run dev` + `curl` cobrindo o fluxo principal (ex.: criar técnico admin → login técnico → abrir OS → aceitar → adicionar material → concluir → avaliar), antes de passar pro próximo módulo. Ao final, abrir o frontend (`npm run dev` em `kovermanutencao-main`) e navegar pelas telas dos módulos da Fase 1 no browser para confirmar ponta a ponta.